手掌检测

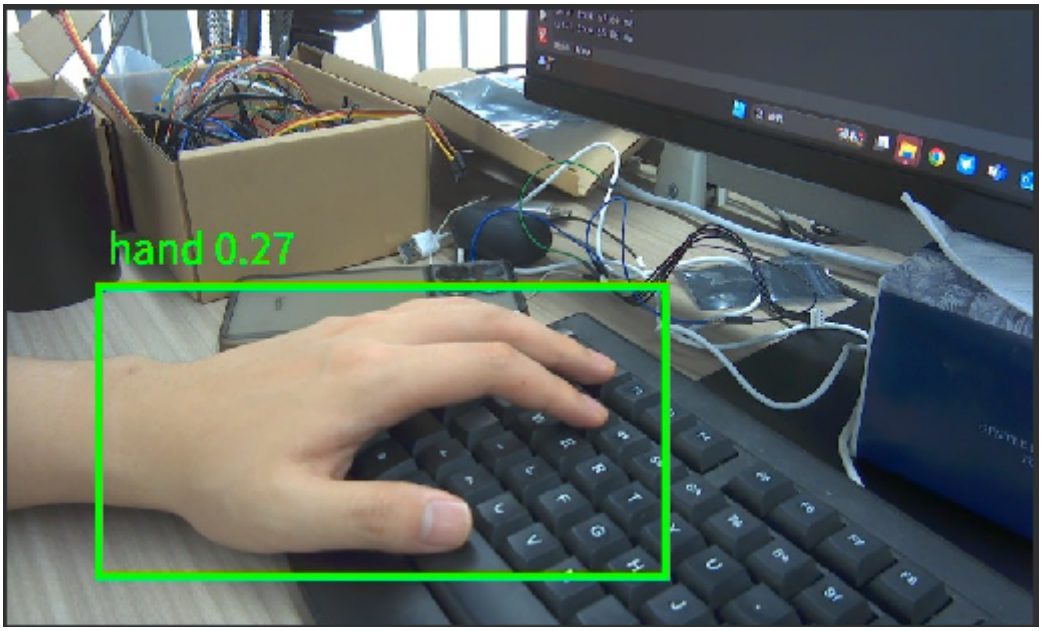
手掌检测
- 例程实验效果
- 代码讲解
  - 流程图
- 手掌检测算法简述
  - 常见的应用场景
  - 算法简述
    - 网络结构
    - 检测原理

例程实验效果

本节我们来学习一下使用K230实现手掌检测的功能。

例程代码在【源码汇总 / 08.Body / 05.hand_detection.py 】

例程运行效果如下，K230会识别出画面中出现的所有手



> 由于人的手部姿态变化较多，为了减少识别不出的情况，识别的阈值设置的不高。这可能会导致把其它不是手的东西识别成手。如果需要更高精度的识别，大家可以自己用K230拍摄手部的照片，标注之后自行训练

> 已添加串口输出功能
>
> 检测到手部后会发送如下格式的串口输出
>
> $x,y,w,h#
>
> 其中'$'代表数据的开头, '#'代表数据的结尾
>
> x，y，w，h是手部检测框的位置（分辨率为640*480）

代码讲解

完整代码请参考【源码汇总 / 08.Body / 04.hand_detection.py】文件

```
if __name__=="__main__":

    display_mode="lcd"
    rgb888p_size = [640,360]

    # 根据显示模式设置显示分辨率
    # Set the display resolution according to the display mode
    if display_mode=="hdmi":
        display_size=[1920,1080]
    else:
        display_size=[640,480]

    # 模型路径
    # Model path
    kmodel_path="/sdcard/kmodel/hand_det.kmodel"

    # 其它参数设置
    # Other parameter settings
    confidence_threshold = 0.2
    nms_threshold = 0.5
    labels = ["hand"]
    anchors = [26,27, 53,52, 75,71, 80,99, 106,82, 99,134, 140,113, 161,172,
245,276]   #anchor设置

    # 初始化PipeLine
    # Initialize PipeLine

 pl=PipeLine(rgb888p_size=rgb888p_size,display_size=display_size,display_mode=di
splay_mode)
    pl.create()

    # 初始化自定义手掌检测实例
    # Initialize custom hand detection instance
    hand_det=HandDetectionApp(kmodel_path,model_input_size=
[512,512],labels=labels,anchors=anchors,confidence_threshold=confidence_threshol
d,nms_threshold=nms_threshold,nms_option=False,strides=
[8,16,32],rgb888p_size=rgb888p_size,display_size=display_size,debug_mode=0)
    hand_det.config_preprocess()

    # 主循环，持续检测并显示结果
    # Main loop, continuously detect and display results
    while True:
        with ScopedTiming("total",1):
            # 获取当前帧数据
            # Get current frame data
            img=pl.get_frame()

            # 推理当前帧
            # Infer current frame
            res=hand_det.run(img)

            # 绘制结果到PipeLine的osd图像
            # Draw results to PipeLine's osd image
```

```
            hand_det.draw_result(pl,res)

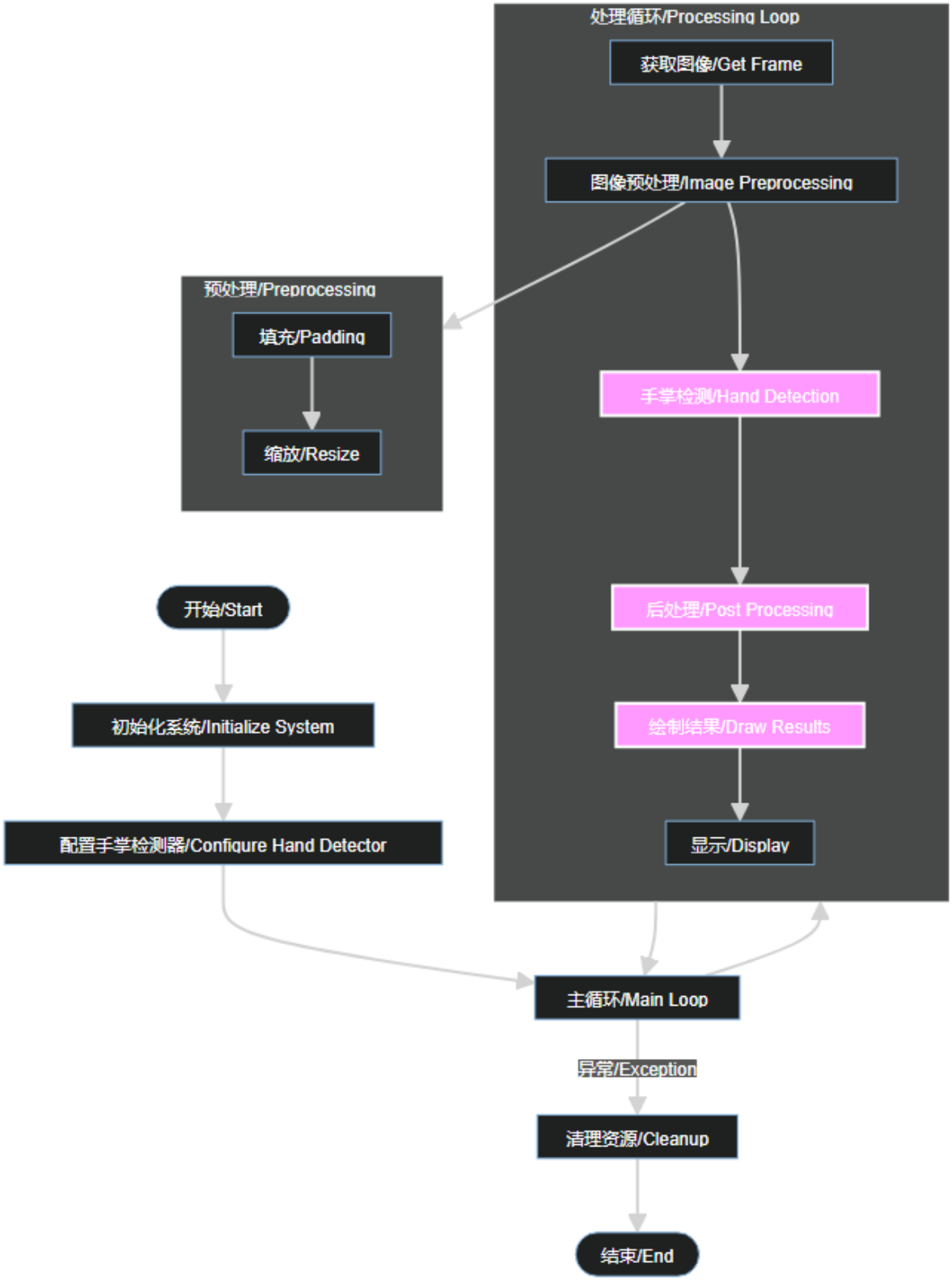
            # 显示当前的绘制结果
            # Display current drawing results
            pl.show_image()

            # 垃圾回收
            # Garbage collection
            gc.collect()

    # 反初始化
    # Deinitialize
    hand_det.deinit()

    # 销毁PipeLine实例
    # Destroy PipeLine instance
    pl.destroy()
```

流程图



手掌检测算法简述

> 手掌检测是后续很多应用场景的基础要求

常见的应用场景

手掌检测算法在现代生活中有许多实用场景

生物识别和身份验证

- 门禁系统
- 移动支付
- 设备解锁

人机交互

- 虚拟现实/增强现实控制
- 智能家居手势控制
- 无触摸显示屏操作

安全监控

- 工业安全(危险区域手势警告)
- 驾驶行为分析
- 公共场所可疑行为检测

医疗健康

- 手部康复训练
- 手术室无接触操作
- 远程医疗手势交互

教育培训

- 手语识别教学
- 乐器演奏指导
- 运动姿势纠正

算法简述

网络结构

特征提取网络

- 输入尺寸为512×512的RGB图像
- 使用多尺度特征提取，有3个特征层级(stride=[8,16,32])
- 不同尺度特征图对应检测不同大小的目标

检测头设计

- 采用多尺度检测策略
- 每个特征图位置预设9组anchors框
- anchors尺寸从小到大分别为:
    - 小尺寸:(26×27, 53×52, 75×71)
    - 中尺寸:(80×99, 106×82, 99×134)
    - 大尺寸:(140×113, 161×172, 245×276)

检测原理

预处理阶段

- 输入图像缩放到512×512
- 保持长宽比例，不足部分padding
- 使用 NCHW格式和双线性插值进行resize

检测过程

- 网络输出 3 个分支，对应 3个不同尺度
- 每个位置预测:
    - 类别概率 (手掌/非手掌)
    - 边界框回归值 (x,y,w,h)
    - 置信度分数

后处理流程

- 置信度阈值过滤(confidence_threshold=0.2)
- 执行NMS(nms_threshold=0.5)去除重叠框
- 边界框解码得到最终检测结果
- 小目标和边缘目标额外过滤:
    - 高度<0.1×显示宽度的框被过滤
    - 宽度和位置不合理的框被过滤